From UIMA CAS to a Video Player: How the Bundestag Pipeline Fits Together

Video-Emotion Recognition project notes

July 10, 2026

1 What this document is for

The Java DUUI pipeline that actually runs face detection, speaker diarization, transcription and the three emotion models is not part of this repository — it lives elsewhere and hands us a single artifact: a UIMA CAS serialized as an `.xmi` file, together with the source video. Everything described here starts from that file. There are three moving parts: a Python parser that turns the CAS into rows in Postgres, the Postgres schema itself, and a small FastAPI + vanilla-JS viewer that plays the video back with everything overlaid in sync. This note walks through all three in the order data actually flows through them.

2 Why the CAS is annoying to parse

A UIMA CAS cannot be read without the typesystem that describes it — an XML document listing every annotation type and its fields. The three typesystem files we were given (`IdentityEmotionTypeSystem.xml`, `MultimodalIdentityTypeSystem.xml`, `EmotionTypeSystem.xml`) turned out to be mostly copies of each other rather than complementary pieces, and between them they still reference a handful of types — `MultimediaElement`, `Embedding`, the bare `MetaData`, `AudioToken` — that none of the three actually define. Those come from external shared typesystem imports that never shipped with the export.

`duui_parser/typesystem.py` deals with this once, up front: `load_merged_typesystem()` concatenates the three files' type descriptions, drops exact duplicates by name, and injects hand-written fallback definitions for the types that are referenced but missing, based on what their fields actually look like on real annotation instances. Anything still undefined after that gets a generic featureless stub so loading succeeds instead of raising.

Loading the CAS itself (`pipeline.py`) uses two non-default flags:

```
cas = load_cas_from_xmi(f, typesystem=merged_typesystem,
                        lenient=True, trusted=True)
```

`lenient=True` means an annotation whose type is not in the merged typesystem at all — the DKPro linguistic layer (Token, Lemma, POS, Dependency, NamedEntity), which this pipeline never populates in a form we use — is silently skipped rather than aborting the whole load. `trusted=True` switches on lxml's huge-tree parsing; a full-session export is a single-line XML document north of 40 MB and lxml's default buffer limit rejects it otherwise.

3 The parser

duui_parser/parsers/ has one module per table family, and they run in a fixed order (parsers/__init__.py):

```
PARSE_STEPS = [
    video, model, person, segment, token,
    embedding, presence, detection, emotion, text_emotion,
]
```

The order is not arbitrary. video runs first because everything else needs the video's id. person runs early because the schema's foreign keys mean a persons row has to exist before anything that references person_id can be inserted, and because it also builds two lookup dictionaries — face_id_to_person_id and voice_id_to_person_id — that every later step needs.

Every module follows the same shape: walk every UIMA view with select_across_views (video-derived and transcript-derived annotations sit on separate sofas in this pipeline, so a plain cas.select() on one view would miss half the data), then run an INSERT ... ON CONFLICT DO NOTHING/UPDATE per instance. The XMI id of each annotation is reused directly as its Postgres primary key, which is what makes re-running the same file idempotent — rerun it twice and you get the same rows, not duplicates. The whole import happens inside one transaction; anything raising partway through rolls the entire thing back, so a half-imported video should never exist.

3.1 Identity resolution

This is the part that took the most debugging. FaceIdentity and VoiceIdentity annotations each carry a bare string id (faceId="face_1", voiceId="SPEAKER_00"). A separate identity:Person annotation is the only thing that actually links those strings to a person, via a pipe-delimited label:

```
<identity:Person xmi:id="32539" personId="person_1"
                 label="voice:SPEAKER_00|face:face_1|score:0.83"/>
```

identity_resolution.py parses that label and, for every downstream annotation that only carries a bare faceId/voiceId (detections, embeddings, presences, emotions), looks it up against the maps built from Person rows. If a given CAS export has no identity:Person annotations at all — which happened with one of the two full-session exports we were given, and not with the other — every one of those lookups returns None for the whole file, regardless of how many FaceIdentity/VoiceIdentity records exist. That is a property of the source export, not a parser bug; the fix was getting a different .xmi that actually had the identity-linking stage run.

The important guarantee here, worth stating explicitly because it is easy to assume otherwise: *a failed identity lookup never drops the row.* Every detection, presence interval and emotion reading gets inserted with whatever person_id it resolved to, including NULL. On the video currently loaded, that is the difference between 1970 face detections total and 1474 of them actually tagged with a person — the other 496 are real, present rows for faces the pipeline saw but never linked to an identity.

4 The database

schema/schema.sql defines fifteen tables, which fall into three groups.

Metadata: videos (one row per source file) and models (provenance — which face/voice/emotion model produced a given row, since several are in play).

Identity: `global_persons` (cross-video identity, almost never populated in practice — cross-video clustering is the exception) and `persons` (per-video identity, the one that actually matters day to day), backed by `face_embeddings`/`voice_embeddings` (native `pgvector` columns, 512-dim and 192-dim), with `presences` recording on-screen/speaking intervals and `face_detections`/`person_detections` carrying the actual per-frame bounding boxes, normalized to $[0, 1]$ so they are resolution independent.

Content and emotion: `segments` covers both camera shots and spoken sentences and, for sentences, carries both a time window and a character-offset window into the transcript — the bridge that lets you map a moment in the video to a span of text. `linguistic_tokens` is one row per spoken word; sentences do not store their own text, so the frontend reconstructs a subtitle by joining every token whose `start_time` falls inside the sentence's window, deliberately not by foreign key, because the token-to-segment FK is not reliably populated by the real annotator. `base_emotions` is the central table: one row per emotion reading, tagged by `modality` (video / audio / text) and `granularity`, with a genuinely different set of populated columns per modality — video rows are per-frame with a real bounding box, audio rows carry a dominance score that text rows never have, and text rows in practice come from two different upstream annotators of differing completeness. `emotion_scores` holds the full label distribution behind each reading's `dominant_label`, and `fused_emotions` holds an optional cross-modal aggregate that is empty for most runs, including the one currently loaded.

A theme worth naming: nearly every foreign key in this schema is nullable on purpose, because real annotator output is incomplete in ways the schema has to tolerate rather than hide. Most `presences` rows have no resolved `person_id`. `fused_emotions` is routinely empty. Text-modality rows sometimes have no `start_time` at all. None of this is a defect to fix; it is what the data actually looks like, and every consumer downstream — the frontend, the query agent — is written to expect it.

5 The frontend

`webapp/server.py` is deliberately thin. It exposes three things: a list of videos, one JSON payload per video containing every sentence, shot, emotion reading and detection for it, and the raw video bytes themselves served with range-request support so seeking works. The client fetches that one payload exactly once per video and never calls the API again during playback — everything after that is a client-side lookup against in-memory arrays.

`webapp/static/app.js` hooks the video element's `timeupdate` event when paused and `requestAnimationFrame` while playing, and on every tick asks two questions of the data it already has:

```
// which rows genuinely span this instant
function coveredBy(rows, t) {
  return rows.filter(r => r.start_time <= t && t <= r.end_time);
}

// which frame-sampled row is closest, within tolerance
function nearestByTime(rows, t, tolerance) {
  // detections are sampled per frame, not continuous,
  // so "covers this instant" doesn't apply to them
}
```

Sentences and time-anchored emotions use the first kind of lookup; face and person detections use the second, since they are sampled at discrete frames rather than spanning a continuous interval. Boxes are drawn on a canvas overlaid on the video element, scaled from the stored $[0, 1]$

coordinates to whatever size the video is actually rendered at, colored by `person_id` through a small fixed palette (grey for anything unresolved), and tagged with a video-emotion label by joining on the shared `frame_index` between a detection and the emotion reading computed from it.

A natural-language "Ask" panel sits above the player and is worth mentioning separately because it is the one piece that talks to an external model rather than just Postgres. It sends the question to Claude along with a hand-written schema description — the semantics above, not just column names, since things like "compare emotions across modalities by valence rather than by label string" are not recoverable from `information_schema` alone — Claude proposes and previews a SQL query through a tool-use loop, and only the final validated query is actually executed, inside a read-only transaction with a statement timeout and a row cap, never trusting rows the model claims to have already seen.

6 Putting it together

Concretely, going from a fresh `.xmi` to a working demo is: `python main.py cas/your_file.xmi` to parse and commit, then `uvicorn webapp.server:app -reload` to serve it. Nothing is ever baked into the video file itself — every overlay is computed at playback time from what is in Postgres, so improving a detection model, re-running identity resolution, or fixing a mislabeled emotion is always just a matter of re-parsing and reloading the page, never re-encoding video.